

M.Sc. Quantitative Finance

---

# MSQF 536 – Machine Learning Techniques

Unit II: Supervised Learning

---

## Practice Sheet

Applied Practical Exercises on Supervised Learning, Regression,  
Loss Functions and Model Evaluation

### Coverage:

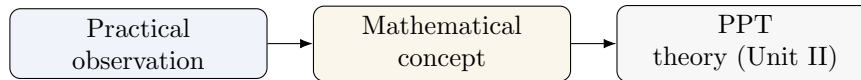
2.1 What is Supervised Learning?    2.2 Framework of Regression    2.3  
Model: The Mathematical Representation

**Instructor:** Pranav T V  
Research Scholar, Department of Statistics  
Pondicherry University  
**Contact:** pranavtvijayan@pondiuni.ac.in

Students should attempt every practical using Python (NumPy, pandas, matplotlib and scikit-learn where required). Written observation spaces are provided after each computational task – use them to connect your numerical result to the theory discussed in class.

## How to Use This Practice Sheet

This sheet contains seven short, self-contained practicals of increasing difficulty. Each practical begins with a small quantitative-finance scenario and a small dataset, followed by numbered tasks. Complete the Python tasks in a Jupyter notebook, then answer the interpretation questions in the space provided. No practical requires more than a small dataset and a few lines of code – the goal is to *discover* the mathematical ideas of Unit II by doing, not to build a large machine-learning project.



## Practical 1 From Data to Prediction

**Learning objective:** Identify input variables and the target variable in a labelled dataset, visualise the input–output relationship, and produce a first, very simple prediction rule. [6 marks]

As introduced in the PPT, suppose we track a stock’s **today’s return** and the **next-day return** for a short recent period. Supervised learning uses such labelled observations  $(x_i, y_i)$  to learn a mapping from inputs to a known target.

**Dataset (10 trading days):**

| Day $i$ | Today’s return $x_i$ (%) | Next-day return $y_i$ (%) |
|---------|--------------------------|---------------------------|
| 1       | 1.2                      | 0.8                       |
| 2       | -0.5                     | -0.2                      |
| 3       | 0.9                      | 1.1                       |
| 4       | -1.3                     | -0.6                      |
| 5       | 2.0                      | 1.4                       |
| 6       | 0.3                      | 0.5                       |
| 7       | -0.8                     | -0.9                      |
| 8       | 1.6                      | 1.0                       |
| 9       | -0.2                     | 0.1                       |
| 10      | 1.0                      | 0.7                       |

**Tasks:**

- 1.1 Using the PPT’s terminology  $D = \{(x_i, y_i)\}_{i=1}^n$ , state clearly which column is the **input variable**  $x_i$  and which is the **target / response variable**  $y_i$  in this problem.
- 1.2 Load the data into a pandas `DataFrame` and produce a scatter plot of  $y_i$  against  $x_i$ .
- 1.3 Propose the simplest possible prediction rule you can think of relating  $y$  to  $x$  (for example, a straight line drawn by eye, or  $\hat{y} = x$ ). Write this rule as a function  $\hat{y}_i = \hat{f}(x_i)$ .
- 1.4 Using your rule, compute  $\hat{y}_i$  for every observation in the table and add it as a new column.

In []:

```
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: build the dataset
data = {
    "x_today_return": [1.2, -0.5, 0.9, -1.3, 2.0, 0.3, -0.8, 1.6, -0.2, 1.0],
    "y_next_return": [0.8, -0.2, 1.1, -0.6, 1.4, 0.5, -0.9, 1.0, 0.1, 0.7]
}
df = pd.DataFrame(data)
print(df.head())

# Step 2: visualise the relationship between X and Y
plt.scatter(df["x_today_return"], df["y_next_return"], color="teal")
plt.xlabel("Today's return, x (%)")
plt.ylabel("Next-day return, y (%)")
plt.title("Input X vs Target Y")
plt.grid(True)
plt.show()

# Step 3: a simple prediction rule  $\hat{y} = f(x)$ 
def simple_rule(x):
    # TODO: propose your own rule and edit this function
    return 0.6 * x

# Step 4: generate predictions
df["y_hat"] = df["x_today_return"].apply(simple_rule)
print(df)
```

Look back at the PPT diagram: *Training data*  $(x_i, y_i) \rightarrow$  *Learning algorithm*  $\rightarrow$  *Learned model*  $\hat{f}(x) \rightarrow$  *Prediction*  $\hat{y}$ . Which part of this diagram did you perform manually in Task 1.3, and which part would normally be automated by a learning algorithm?

---

---

---

## Practical 2 Understanding Prediction Error

**Learning objective:** Compute prediction errors  $e_i = y_i - \hat{y}_i$  and squared errors, and connect them to the idea of a loss function  $L(y_i, \hat{y}_i)$ . [6 marks]

The PPT states: for the actual next-day return  $y = 0.020$  and a model prediction  $\hat{y} = 0.012$ , the prediction error is  $e = y - \hat{y} = 0.008$ . We now repeat this calculation for a small set of observations and compare it against squared error.

**Dataset:** actual and predicted next-day returns (in decimal form, not %).

| Day $i$ | Actual $y_i$ | Predicted $\hat{y}_i$ |
|---------|--------------|-----------------------|
| 1       | 0.020        | 0.015                 |
| 2       | -0.010       | 0.000                 |
| 3       | 0.030        | 0.025                 |
| 4       | -0.005       | -0.012                |
| 5       | 0.012        | 0.004                 |

### Tasks:

- 2.1 By hand, compute the prediction error  $e_i = y_i - \hat{y}_i$  for observation  $i = 1$  and  $i = 4$  only, showing your working.
- 2.2 Using Python, compute  $e_i$  for *all* observations in the table.
- 2.3 Compute the squared error  $e_i^2 = L_{SE}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$  for every observation.
- 2.4 Identify, from the table, which single observation contributes the **largest** squared error, and state its value.

In []:

```
import numpy as np
import pandas as pd

y      = np.array([0.020, -0.010, 0.030, -0.005, 0.012])
y_hat  = np.array([0.015,  0.000, 0.025, -0.012, 0.004])

# Task 2.2: prediction error  $e_i = y_i - y_{\hat{i}}$ 
error = y - y_hat

# Task 2.3: squared error  $(y_i - y_{\hat{i}})^2$ 
squared_error = error ** 2

result = pd.DataFrame({
    "y": y, "y_hat": y_hat,
    "error": error, "squared_error": squared_error
})
print(result)

# Task 2.4: locate the largest squared error
worst = result["squared_error"].idxmax()
print("Largest squared error at observation:", worst)
```

The PPT defines the squared-error loss as  $L_{SE}(y, \hat{y}) = (y - \hat{y})^2$ . Why does squaring the error (rather than, say, leaving it signed, or simply summing the raw errors) make sense as a way to measure how “bad” a prediction is? Refer to your result from Task 2.4 in your answer.

---

---

---

## Practical 3 Comparing Prediction Models Using MSE

**Learning objective:** Compute the Mean Squared Error,  $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ , for two competing prediction rules and use it to decide which rule is better, mirroring the empirical-risk formulation  $\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_i L(y_i, f(x_i; \theta))$ . [8 marks]

An analyst proposes **two** simple rules for predicting next-day return  $y$  from today's return  $x$ :

Model A:  $\hat{y}_i = 0$

Model B:  $\hat{y}_i = x_i$

Model A simply predicts “no change” every day; Model B predicts that tomorrow's return will equal today's return exactly. We would like to know which model fits the observed data better.

**Dataset:** reuse the data from Practical 1 (Section 1).

**Tasks:**

**3.1** For every observation, compute the predictions of Model A and Model B.

**3.2** Compute the squared error of each model for every observation.

**3.3** Compute  $\text{MSE}_A$  and  $\text{MSE}_B$  using the formula  $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ .

**3.4** State which model has the lower MSE on this dataset.

**3.5** Suggest a *third* rule,  $\hat{y}_i = \beta x_i$  for some constant  $\beta$  of your choice, and check whether it achieves an even lower MSE than Models A and B.

In []:

```
import numpy as np

x = np.array([1.2, -0.5, 0.9, -1.3, 2.0, 0.3, -0.8, 1.6, -0.2, 1.0])
y = np.array([0.8, -0.2, 1.1, -0.6, 1.4, 0.5, -0.9, 1.0, 0.1, 0.7])

# Model A: y_hat = 0
y_hat_A = np.zeros_like(y)

# Model B: y_hat = x
y_hat_B = x.copy()

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

mse_A = mse(y, y_hat_A)
mse_B = mse(y, y_hat_B)
print("MSE Model A:", mse_A)
print("MSE Model B:", mse_B)

# Task 3.5: try your own beta
beta = 0.6 # TODO: choose a value and experiment
y_hat_C = beta * x
mse_C = mse(y, y_hat_C)
print("MSE Model C (beta = {}):".format(beta), mse_C)
```

Which of Models A, B and C achieved the lowest MSE? Relate this comparison to the empirical-risk formulation  $\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_i L(y_i, f(x_i; \theta))$  – what role is  $\beta$  playing in this expression, and what is the “learning algorithm” effectively trying to do when it searches over values of  $\beta$ ?

---

---

---

## Practical 4 Regression versus Classification

**Learning objective:** Distinguish a regression target from a classification target by constructing both from the same raw data, following the PPT's example  $\hat{C}_{t+1} = 1$  if  $r_{t+1} > 0$ , else 0. **[6 marks]**

A risk desk wants to know two different things about the same stock: (a) *how much* the return will be tomorrow, and (b) *whether* tomorrow's return will be positive or negative. As the PPT notes, the same predictors can be used for both tasks – the difference lies in the target variable.

**Dataset:** next-day returns for 10 days.

| Day $i$ | $r_{t+1}$ (%) |
|---------|---------------|
| 1       | 0.8           |
| 2       | -0.2          |
| 3       | 1.1           |
| 4       | -0.6          |
| 5       | 1.4           |
| 6       | 0.5           |
| 7       | -0.9          |
| 8       | 1.0           |
| 9       | 0.1           |
| 10      | 0.7           |

### Tasks:

- 4.1 State, in one line, why predicting  $r_{t+1}$  directly is a **regression** problem (refer to the nature of the target variable, as in the PPT's Regression/Classification table).
- 4.2 Construct a new column  $C_{t+1}$  using the PPT's rule  $C_{t+1} = 1$  if  $r_{t+1} > 0$ , and  $C_{t+1} = 0$  if  $r_{t+1} \leq 0$ .
- 4.3 State why predicting  $C_{t+1}$  is a **binary classification** problem.
- 4.4 Count how many days are labelled  $C_{t+1} = 1$  and how many are labelled  $C_{t+1} = 0$ .

In []:

```
import pandas as pd
import numpy as np

r_next = pd.Series([0.8, -0.2, 1.1, -0.6, 1.4, 0.5, -0.9, 1.0, 0.1, 0.7])

# Task 4.2: build the classification target
C_next = np.where(r_next > 0, 1, 0)

df = pd.DataFrame({"r_next": r_next, "C_next": C_next})
print(df)

# Task 4.4: count each class
print(df["C_next"].value_counts())
```



The PPT states: “*The same financial predictors can be used in both regression and classification. The key difference is the target and therefore the statistical model and loss function used.*” In your own words, explain why the squared-error loss  $L_{SE}(y, \hat{y}) = (y - \hat{y})^2$  used in Practicals 2–3 is natural for  $r_{t+1}$  but is *not* an obviously sensible way to score a prediction of the class label  $C_{t+1}$ .

---

---

---

## Practical 5 Training Data versus Unseen/Test Data

**Learning objective:** Split a dataset into training and test observations, fit a very simple model using only the training data, and compare training error against test error, following the PPT's Training/Test/Generalization diagram. [8 marks]

As the PPT warns: “A model that predicts historical stock returns extremely well may fail on future market data because the model has captured historical noise rather than a stable predictive relationship.” We now check this directly using the returns from Practical 1, respecting time order (no shuffling).

**Dataset:** reuse the 10 days of  $(x_i, y_i)$  from Practical 1 (Section 1). Use days 1–7 as the **training set** and days 8–10 as the **test set**, preserving the original time order (as emphasised for financial data in the PPT).

**Tasks:**

- 5.1 Split the data into training (days 1–7) and test (days 8–10) subsets.
- 5.2 Using *only the training data*, compute the simplest possible model: the mean of  $y$  in the training set,  $\hat{y} = \bar{y}_{\text{train}}$ .
- 5.3 Using this single fitted value  $\hat{y}$ , compute the **training MSE** (the MSE evaluated on the training observations).
- 5.4 Using the *same* fitted value  $\hat{y}$  (do not re-fit), compute the **test MSE** (the MSE evaluated on the test observations).
- 5.5 Compare the two MSE values.

In []:

```
import numpy as np

x = np.array([1.2, -0.5, 0.9, -1.3, 2.0, 0.3, -0.8, 1.6, -0.2, 1.0])
y = np.array([0.8, -0.2, 1.1, -0.6, 1.4, 0.5, -0.9, 1.0, 0.1, 0.7])

# Task 5.1: preserve time order -- no shuffling
x_train, y_train = x[:7], y[:7]
x_test, y_test = x[7:], y[7:]

# Task 5.2: fit the simplest possible model on the TRAINING set only
y_hat = np.mean(y_train)
print("Fitted value (trained on days 1-7):", y_hat)

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

# Task 5.3: training error
train_mse = mse(y_train, y_hat)

# Task 5.4: test error, using the SAME fitted value
test_mse = mse(y_test, y_hat)

print("Training MSE:", train_mse)
print("Test MSE:      ", test_mse)
```

Is the test MSE larger, smaller, or about the same as the training MSE? The PPT distinguishes *training error* (calculated on observations used to fit the model) from *test error* (calculated on unseen observations), and defines *generalization* as the model's ability to perform well on unseen data. Based on your two MSE values, would you say this simple model generalizes well?

---

---

---

## Practical 6 Fitting and Interpreting a Simple Regression Model

**Learning objective:** Fit the parametric linear model  $Y = \beta_0 + \beta_1 X + \varepsilon$  to data, interpret the estimated coefficients  $\hat{\beta}_0, \hat{\beta}_1$ , and use the fitted model  $\hat{y}_{\text{new}} = \hat{f}(x_{\text{new}})$  to predict a new observation. **[10 marks]**

The PPT's parametric regression framework specifies  $Y_i = \beta_0 + \beta_1 X_{i1} + \varepsilon_i$  for a single predictor. We now fit this model to the training data from Practical 5 and interpret the coefficients in a finance context.

**Dataset:** training subset  $(x_i, y_i)$ ,  $i = 1, \dots, 7$ , from Practical 5.

**Tasks:**

- 6.1** Fit the linear model  $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$  to the **training** data using `LinearRegression` from `scikit-learn`.
- 6.2** Report the estimated intercept  $\hat{\beta}_0$  and slope  $\hat{\beta}_1$ .
- 6.3** In one sentence, interpret  $\hat{\beta}_1$  using the PPT's description of  $\beta_j$  as “the change in the conditional mean associated with  $X_j$ ”.
- 6.4** A new day arrives with today's return  $x_{\text{new}} = 1.5\%$ . Use the fitted model to predict  $\hat{y}_{\text{new}}$ .
- 6.5** Compute the residuals  $e_i = y_i - \hat{y}_i$  on the training data and verify that your model's training MSE is no worse than the constant-mean model of Practical 5.

In []:

```
import numpy as np
from sklearn.linear_model import LinearRegression

x_train = np.array([1.2, -0.5, 0.9, -1.3, 2.0, 0.3, -0.8]).reshape(-1, 1)
y_train = np.array([0.8, -0.2, 1.1, -0.6, 1.4, 0.5, -0.9])

# Task 6.1: fit the parametric linear model y = beta0 + beta1 * x
model = LinearRegression()
model.fit(x_train, y_train)

# Task 6.2: estimated coefficients
beta0 = model.intercept_
beta1 = model.coef_[0]
print("beta0_hat (intercept):", beta0)
print("beta1_hat (slope):", beta1)

# Task 6.4: predict a new observation
x_new = np.array([[1.5]])
y_new_hat = model.predict(x_new)
print("Predicted next-day return for x_new = 1.5%:", y_new_hat[0])

# Task 6.5: residuals and training MSE
y_hat_train = model.predict(x_train)
residuals = y_train - y_hat_train
train_mse_linear = np.mean(residuals ** 2)
print("Residuals:", residuals)
print("Training MSE (linear model):", train_mse_linear)
```

Compare the training MSE obtained here with the training MSE of the constant-mean model from Practical 5. The PPT notes that  $\hat{\beta} = \arg \min_{\beta} \frac{1}{n} \sum_i [y_i - f(x_i; \beta)]^2$ , i.e. least squares chooses parameters to minimize squared error on the training data. Does your result confirm that a model with an extra parameter ( $\beta_1$ ) can fit the training data at least as well as a model with only one parameter ( $\beta_0$  alone)?

---

---

---

## Practical 7 Empirical Risk and the Choice of Loss Function

**Learning objective:** Compute the empirical risk  $\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i; \theta))$  under two different loss functions (MSE and MAE) and observe how an extreme observation affects each differently. [8 marks]

The PPT notes that financial returns can contain extreme observations caused by market crashes, and that MAE or Huber loss may be preferred when a few extreme returns dominate the MSE. We test this using the fitted model from Practical 6, after adding one extreme “crash day” to the test period.

**Dataset:** test set from Practical 5,  $x_{\text{test}} = (1.6, -0.2, 1.0)$ ,  $y_{\text{test}} = (1.0, 0.1, 0.7)$ , with one additional crash-day observation appended:  $x = -2.5$ ,  $y = -4.2$ .

**Tasks:**

- 7.1 Using the model fitted in Practical 6, predict  $\hat{y}_i$  for all four test observations (including the crash day).
- 7.2 Compute the empirical risk under squared-error loss,  $\hat{R}_{\text{MSE}} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$ .
- 7.3 Compute the empirical risk under absolute-error loss,  $\hat{R}_{\text{MAE}} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$ .
- 7.4 Remove the crash-day observation and recompute both  $\hat{R}_{\text{MSE}}$  and  $\hat{R}_{\text{MAE}}$  on the remaining three observations.
- 7.5 Compare, in each case (with and without the crash day), how much  $\hat{R}_{\text{MSE}}$  changed versus how much  $\hat{R}_{\text{MAE}}$  changed.

In []:

```
import numpy as np
from sklearn.linear_model import LinearRegression

# model re-fitted from Practical 6 (training data)
x_train = np.array([1.2, -0.5, 0.9, -1.3, 2.0, 0.3, -0.8]).reshape(-1, 1)
y_train = np.array([0.8, -0.2, 1.1, -0.6, 1.4, 0.5, -0.9])
model = LinearRegression().fit(x_train, y_train)

# test set + one crash-day observation
x_test = np.array([1.6, -0.2, 1.0, -2.5]).reshape(-1, 1)
y_test = np.array([1.0, 0.1, 0.7, -4.2])

y_hat = model.predict(x_test)

def empirical_risk_mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def empirical_risk_mae(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

# Task 7.2 - 7.3: with the crash day included
R_mse_with = empirical_risk_mse(y_test, y_hat)
R_mae_with = empirical_risk_mae(y_test, y_hat)

# Task 7.4: without the crash day
R_mse_without = empirical_risk_mse(y_test[:3], y_hat[:3])
R_mae_without = empirical_risk_mae(y_test[:3], y_hat[:3])
```

```
print("With crash day    -> MSE:", R_mse_with, " MAE:", R_mae_with)
print("Without crash day -> MSE:", R_mse_without, " MAE:", R_mae_without)
print("MSE increased by factor:", R_mse_with / R_mse_without)
print("MAE increased by factor:", R_mae_with / R_mae_without)
```

Which loss function, MSE or MAE, changed *proportionally more* once the crash day was included? The PPT states that MSE “strongly penalizes large errors” while MAE is “more robust to outliers.” Does your empirical risk calculation support this statement? Also recall that  $\hat{R}(\theta)$  is only an estimate, computed from a finite sample, of the true expected risk  $R(f) = E_{(X,Y)}[L(Y, f(X))]$  – why might a single unusual crash day make  $\hat{R}(\theta)$  a poor estimate of  $R(f)$  here?

---

---

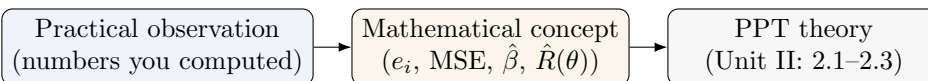
---

---

## Connecting the Practicals to Unit II Theory

The table below summarises how each practical activity you performed maps onto the formal framework presented in the PPT.

| Practical | What you did                                                                                                        | PPT concept / notation                                                                                                         |
|-----------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1         | Identified $x_i, y_i$ , plotted the relationship, guessed a rule $\hat{y} = \hat{f}(x)$                             | $D = \{(x_i, y_i)\}$ , learning pipeline $(x_i, y_i) \rightarrow \hat{f} \rightarrow \hat{y}$                                  |
| 2         | Computed $e_i = y_i - \hat{y}_i$ and $e_i^2$                                                                        | Prediction error, squared-error loss $L_{SE}(y, \hat{y}) = (y - \hat{y})^2$                                                    |
| 3         | Compared MSE of competing rules, tried different $\beta$                                                            | $MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$ ; $\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_i L(y_i, f(x_i; \theta))$ |
| 4         | Built a continuous target and a binary target from the same returns                                                 | Regression vs. classification; $C_{t+1} = \mathbb{K}\{r_{t+1} > 0\}$                                                           |
| 5         | Split data by time, fit on train, evaluated on train <i>and</i> test                                                | Training error vs. test error; generalization                                                                                  |
| 6         | Fitted $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$ , interpreted $\hat{\beta}_1$ , predicted a new $x_{\text{new}}$ | Parametric regression $Y = \beta_0 + \beta_1 X + \varepsilon$ ; least squares $\hat{\beta} = \arg \min_{\beta} R_n(\beta)$     |
| 7         | Computed empirical risk under MSE and MAE, with and without an outlier                                              | Empirical risk $\hat{R}(\theta) = \frac{1}{n} \sum_i L(y_i, f(x_i; \theta))$ vs. population risk $R(f) = E[L(Y, f(X))]$        |



By the end of this sheet you should be able to say: “*I just did, by hand and in code, what the PPT describes mathematically.*”

### Instructor Reference / Expected Learning Outcome

- **Practical 1:** Students should correctly identify  $x$  as the input and  $y$  as the target, and recognise that their by-eye rule is an informal analogue of  $\hat{f}$ .
- **Practical 2:** Students should see that squared error penalises the largest deviation (observation 2) most heavily.
- **Practical 3:** On this dataset, Model B ( $\hat{y} = x$ ) should outperform Model A ( $\hat{y} = 0$ ); an intermediate  $\beta < 1$  typically improves on both, motivating least squares.
- **Practical 4:** Students should articulate that the target’s nature (continuous vs. two categories) – not the predictors – determines whether the problem is regression or classification.
- **Practical 5:** With only seven training points, test MSE will typically differ noticeably from training MSE; the direction is dataset-dependent and is itself a useful discussion point about small samples.
- **Practical 6:** The fitted slope  $\hat{\beta}_1$  should be positive (momentum-like relationship in this constructed dataset); training MSE of the linear model should not exceed that of the constant-mean model.



- **Practical 7:**  $\hat{R}_{\text{MSE}}$  should increase by a much larger factor than  $\hat{R}_{\text{MAE}}$  once the crash day is included, illustrating loss robustness and the gap between empirical risk and true risk.